

Guia de Estudo — InvestiGator

Da estaca zero à defesa: aprender a tese como se a tivesse escrito

Henrique J. S. Santos

Orientador: Prof. Luís Gomes Coorientador: Rafael Silva

MEIA, Instituto Superior de Engenharia do Porto

Material de preparação para a defesa

Como usar este guia

- Foi feito para quem **não sabe nada de IA**. Começa do zero.
- **Uma ideia por slide**. Primeiro a *intuição* (com analogias), só depois o detalhe.
- Lê **pela ordem**: cada parte prepara a seguinte.
- No fim, deves conseguir responder com confiança a "porquê isto?" em cada passo da tese.

Estuda este guia a par com a própria tese e com os slides de defesa (slides/).

O percurso

- 1 IA do zero (só o que a tese usa)
- 2 O problema e a contribuição
- 3 O sistema (dados, partes, fluxo)
- 4 Os dados, a olho
- 5 O código, módulo a módulo
- 6 Workflow ponta a ponta
- 7 Avaliação e decisões
- 8 Perguntas do júri

A tese em três frases (o teu *pitch*)

Se só decorares isto, já defendes a ideia central

- ❶ Construí o **InvestiGator**, um sistema que avisa um investidor de retalho quando há um *movimento de mercado invulgar* ou uma *notícia relevante*, e **explica** cada alerta de forma rastreável (evento, raciocínio, fontes, precedentes históricos).
- ❷ A contribuição é de **Engenharia de IA**: não inventei algoritmos; *integrei, apliquei e avaliei criticamente* componentes existentes num sistema funcional, explicável e reproduzível.
- ❸ **Não prevejo preços**. Mostro o que aconteceu *depois* de notícias parecidas no passado, como evidência, com honestidade sobre as limitações.

O que é Inteligência Artificial? E *Machine Learning*?

Inteligência Artificial (IA)

Fazer com que um computador resolva tarefas que pareceriam precisar de "inteligência" humana (decidir, reconhecer, recomendar).

Machine Learning (ML)

Em vez de escrever *regras à mão*, o computador *aprende padrões a partir de dados*.

Analogia

Regras à mão: explicar a alguém, passo a passo, como reconhecer um gato.

ML: mostrar mil fotos de gatos e deixar a pessoa aprender o padrão sozinha.

Nesta tese, a "aprendizagem" já vem **pronta** (modelos pré-treinados). Ver o slide seguinte.

O que esta tese É — e o que **NÃO** é (muito importante)

O que usa

- Um **embedder de texto pré-treinado** (SBERT) — só para *usar*, não para treinar.
- **Estatística** simples (z-score: média e desvio-padrão).
- **Similaridade do cosseno** (comparar vetores).
- **Event study** (aritmética de retornos).
- **UM modelo treinado por mim**: a *triagem de materialidade* (regressão logística; RQ4). Simples de propósito — ver a parte da Avaliação.
- **Explicabilidade** (mostrar a evidência).

O que **NÃO** usa (e porquê)

- ✗ Treinar redes neurais profundas (épocas, backprop, CNNs)
- ✗ Visão computacional, imagens, segmentação
- ✗ Prever preços ou direção (nunca!)

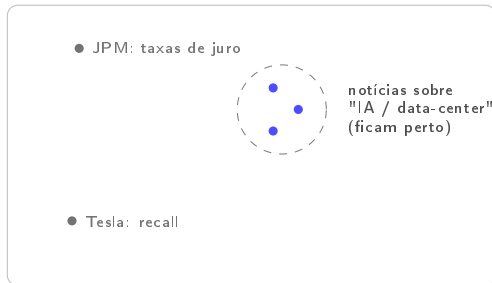
Porquê? A contribuição principal é **integrar componentes existentes**; o modelo que treino é pequeno, transparente e avaliado contra baselines — não é deep learning, é ML clássico defensável.

Se o júri perguntar por treino: "treino UM classificador de triagem (RQ4), com protocolo temporal e anti-lookahead testado — e reporto que a baseline de volatilidade ganhou."

Ideia-chave 1: transformar texto em *números* (*embeddings*)

Um computador não "lê" palavras; compara *números*. Um **embedding** transforma cada título de notícia num **vetor** (uma lista de números = um ponto num espaço).

A magia: títulos com **significado parecido** ficam **perto** uns dos outros nesse espaço.



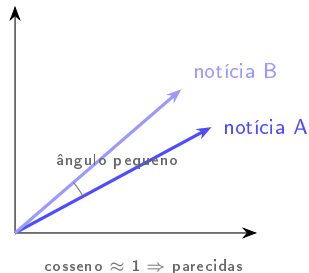
Porquê isto importa: para encontrar notícias passadas *parecidas* com uma nova, basta procurar os pontos *mais próximos*.

Como medir "perto"? *Similaridade do cosseno*

Mede-se o **ângulo** entre dois vetores:

- ângulo pequeno $\Rightarrow$ cosseno ≈ 1 (muito parecidos);
- ângulo reto $\Rightarrow$ cosseno ≈ 0 (não relacionados).

Usa-se o ângulo (e não a distância) porque só interessa a *direção* do significado, não o tamanho do vetor.

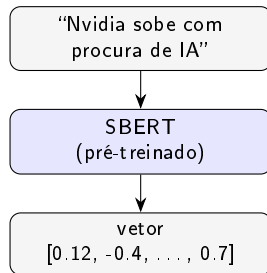


De onde vêm os vetores? *SBERT* (e *Transformer*), em intuição

SBERT (Sentence-BERT) é um modelo **já treinado** por outros, que lê uma frase e devolve o seu vetor de significado. O **Transformer** é a arquitetura que o tornou possível (lê a frase inteira e "presta atenção" às palavras que importam).

Nesta tese usamo-lo pronto (inferência): damos-lhe um título, recebemos o vetor. Não o treinamos.

Porquê SBERT e não um LLM gigante? Capta significado, é **barato**, **reproduzível** e **gratuito**, e dá vetores diretamente comparáveis.



Ideia-chave 2: estatística para "isto é invulgar?"

Média (μ): o valor típico dos últimos dias.

Desvio-padrão (σ): o quanto costuma oscilar (a *volatilidade*).

z-score: a quantos desvios-padrão está o movimento de hoje:

$$z = \frac{r - \mu}{\sigma}$$

Normalizar pela volatilidade é o truque: a mesma queda de $-3,2\%$ é dramática numa ação calma e banal numa volátil.

Exemplo (da tese)

Queda de $-3,2\%$ hoje:

- ação **calma** ($\sigma = 0,40\%$): $z \approx -8,1 \Rightarrow$ anomalia clara
- ação **volátil** ($\sigma = 1,50\%$): $z \approx -2,2 \Rightarrow$ dia normal

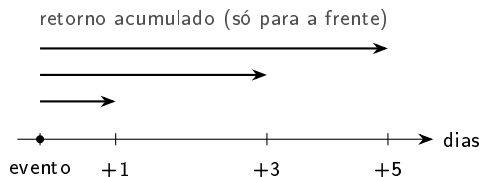
Um limiar fixo em % trataria as duas igual; o z-score não.

Ideia-chave 3: *retornos e event study* (medir o impacto, sem ver o futuro)

Log-return: a variação diária do preço (em %).

Event study: depois de uma notícia, medimos o retorno acumulado a **+1, +3 e +5 dias** a partir do *fecho* do primeiro dia de negociação.

Anti-lookahead (regra de ouro de honestidade): só olhamos para *depois* do evento. Nunca usamos o futuro para "prever". Medimos um *resultado*, não uma previsão.



Ideia-chave 4: *recuperação* e como se mede (precision@k)

Recuperação (Information Retrieval): ordenar notícias passadas pela semelhança a uma nova e mostrar as ***k* melhores** (ex.: $k = 5$).

precision@k: das k recuperadas, que fração é *relevante*? Usamos "mesmo setor" como proxy automático de relevância.

Comparamos sempre com **baselines** para o número ter significado:

- **aleatório** (o "chão" sem perícia),
- **recência** (só as mais recentes),
- **lexical** (sobreposição de palavras).

Intuição

"Das 5 notícias passadas que mostrei, quantas eram mesmo do mesmo tema?"

Bater o aleatório e o lexical é a prova de que os *embeddings* acrescentam valor real.

Ideia-chave 5: explicabilidade (XAI) e confiança

Transparente vs caixa-preta: ou o modelo é interpretável por desenho, ou é opaco e explicamo-lo a *posteriori* (LIME/SHAP). O InvestiGator escolhe **transparente por desenho**.

Fidelidade: a explicação reflete *exatamente* o que o sistema calculou (não é uma aproximação).

Confiança calibrada: queremos que o utilizador confie *na medida certa* — nem ignorar um bom aviso, nem confiar cegamente. Por isso mostramos *evidência verificável*, não uma narrativa.

Raciocínio Baseado em Casos (CBR)

A ideia familiar por trás do motor: como um médico que se lembra de *doentes parecidos* do passado.

Recuperar casos semelhantes → **reutilizar** o que aconteceu como evidência. Paramos aqui (não "prevemos").

Glossário (os termos que vais ouvir)

- **Embedding**: texto $\rightarrow$ vetor de significado.
- **Cosseno**: medida de semelhança (ângulo).
- **SBERT**: modelo pré-treinado que cria embeddings de frases.
- **z-score**: nº de desvios-padrão face à norma recente.
- **Volatilidade**: o quanto uma ação costuma oscilar (σ).
- **Event study**: medir o retorno após um evento.
- **Anti-lookahead**: nunca usar o futuro.
- **precision@k**: acerto nas k melhores.
- **Baseline**: método simples de comparação.
- **XAI / fidelidade**: explicação fiel ao cálculo.
- **CBR**: raciocínio por casos análogos.

Tens agora as bases. As próximas partes aplicam *exatamente* estes conceitos à tese.

Já tens as **bases de IA** de que esta tese precisa.

A seguir: o problema e a contribuição, depois o sistema (dados, partes, fluxo), o código e a avaliação.

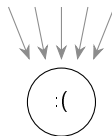
Para quem é? E qual é o problema?

Investidor de retalho: uma pessoa comum que investe (não um profissional).

O problema é a **sobrecarga de informação**: milhares de ações mexem-se e as notícias chegam sem pausa. As ferramentas que ajudam a interpretar isto são **institucionais, opacas, ou ambas**.

E este investidor é guiado pela **atenção**: reage a movimentos extremos e a notícias salientes. Logo, faz sentido intervir *exatamente nesses momentos*, com **contexto e explicação**.

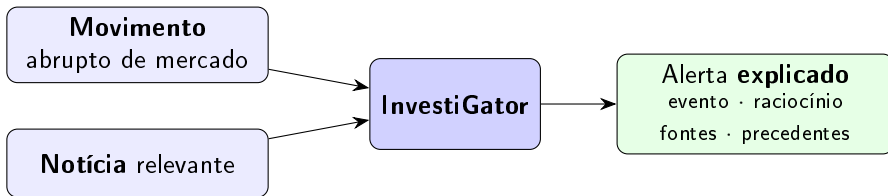
milhares de preços
+ notícias
sem pausa



investidor sozinho

sem as ferramentas que bancos e fundos têm

A resposta: dois gatilhos, sempre com explicação



O alerta não é uma notificação seca: traz a **cadeia de raciocínio completa**, que o investidor pode **seguir e verificar**.

O que faz — e o que **não** faz

Faz

- Deteta movimentos invulgares e explica-os.
- Para uma notícia, mostra **precedentes históricos** reais e o impacto que tiveram.
- Expõe toda a evidência (rastreável).

Não faz (por desenho)

- ✗ Prever preços.
- ✗ Trading automático / conselho.
- ✗ APIs pagas (só gratuitas).

É uma escolha de **honestidade e defensibilidade**: medir o passado como evidência, nunca prometer o futuro.

A contribuição: *Engenharia* de IA

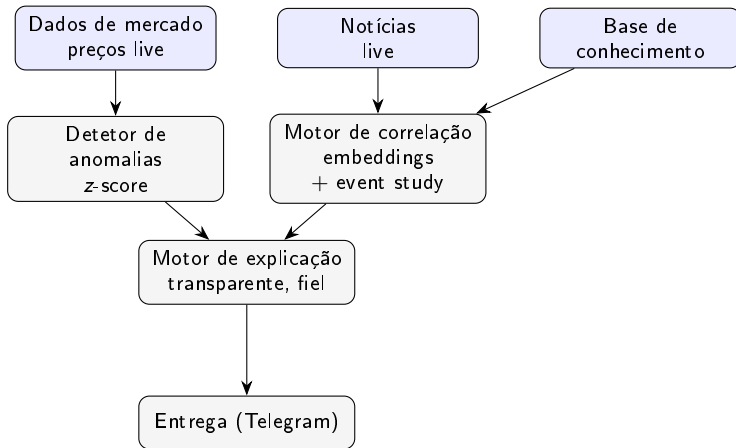
A contribuição **não** é inventar algoritmos. É **integrar, aplicar e avaliar criticamente** componentes existentes num sistema funcional, explicável e reproduzível. Três contribuições concretas:

- 1 Uma **metodologia** reproduzível de correlação notícia→impacto (embeddings + event study, sem lookahead).
- 2 O **InvestiGator**, um sistema de alertas **explicável** cuja cadeia de raciocínio é toda exposta ao utilizador e *fiel por construção*.
- 3 Uma **avaliação crítica e honesta** de cada componente em dados reais, contra baselines, com limitações reportadas.

Defesa em 1 frase

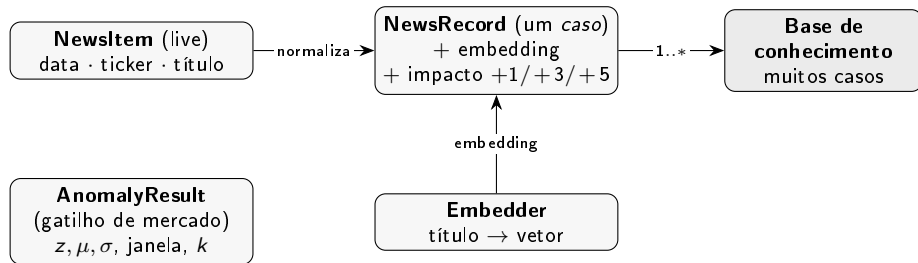
"Usar modelos e ferramentas existentes é o trabalho de engenharia: o valor está no sistema coerente, explicável e reproduzível, não num algoritmo novo."

Arquitetura num relance



Componentes de **responsabilidade única**, ligados por um **esquema comum**. Dois gatilhos convergem num único passo de explicação antes de enviar.

O modelo de dados (os objetos e as relações)



Ideia central: um *caso* = uma notícia passada + o que aconteceu ao preço a seguir. **NewsItem** (live) e **NewsRecord** (histórico) partilham o mesmo esquema, por isso são diretamente comparáveis.

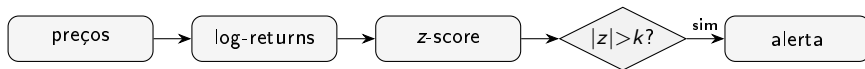
Os componentes e o que cada um faz

Componente	Responsabilidade	Entrada → saída
Dados de mercado	preços, log-returns	ticker → retornos diários
Notícias	obter e normalizar	ticker → NewsItem(s)
Embedder	título → vetor	título → embedding
Base de conhecimento	guardar/recuperar casos	título → top- <i>k</i> casos
Detetor de anomalias	sinalar movimento invulgar	retornos → AnomalyResult
Motor de correlação	cosseno + event study	título, KB → precedentes + impacto
Motor de explicação	montar o alerta fiel	objetos → texto do alerta
Entrega	enviar ao utilizador	texto → Telegram

Cada componente tem **uma** responsabilidade: é o que permite testar offline e trocar peças.

Os dois gatilhos, lado a lado

Gatilho de mercado:



Gatilho de notícia:



Ambos terminam no **motor de explicação**, que monta o alerta a partir dos objetos calculados.

Que dados? Duas camadas

Histórica (offline)

FNSPID: notícias alinhadas a preços diários.

Data card: 15 ações grandes, 5 setores (tecnologia, banca, energia, saúde, consumo), 2018–2023. Licença CC BY-SA 4.0 (atribuição obrigatória).

Live

Preços (yfinance) + notícias (Finnhub/RSS), só APIs **gratuitas**.

A avaliação usa um corpus **real e recente** (3 714 títulos) construído pelo mesmo processo; o FNSPID multi-ano fica como trabalho futuro.

As duas camadas partilham o **mesmo esquema** (data, ticker, título) $\Rightarrow$ o novo é comparável com o histórico.

A estrutura de pastas (o repositório)

Onde vive cada coisa:

DIMEIA/

thesis/	a dissertacao (LaTeX, 6 capitulos)
investigator/	o sistema (um pacote por componente)
data/samples/	amostras pequenas (CSV, JSONL)
scripts/	dados, figuras, avaliacao, build
slides/	slides de defesa + este guia
docs/	documentacao e provas de rigor

Os dados grandes **não** são versionados (recriados por script); só ficam pequenas amostras.

Os dados a olho: notícias (CSV) e um caso (JSON)

Notícias de entrada (news_sample.csv) — três colunas:

```
date,ticker,headline
2023-01-09,AAPL,Apple expands services revenue as iPhone demand...
2023-05-25,NVDA,Nvidia guidance surges on soaring demand for AI...
```

Um "caso" na base de conhecimento (kb_sample.jsonl, uma linha):

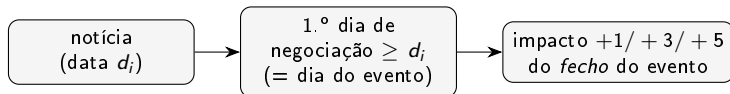
```
{ "date": "2023-01-09", "ticker": "AAPL",
  "headline": "Apple expands services revenue...",
  "impacts": { "1": 0.0045, "3": 0.0250, "5": 0.0445 },
  "embedding": [0.0, 0.0, ..., 0.333]  // 64 numeros }
```

O JSON de um caso, campo a campo

- **date, ticker, headline**: a notícia (igual ao esquema live).
- **impacts**: o que o preço fez **depois** — retorno acumulado a +1, +3 e +5 dias (ex.: +0,45%, +2,50%, +4,45%). *Esta é a "etiqueta" de evidência.*
- **embedding**: o vetor de significado do título (aqui 64 números do baseline; com SBERT seriam centenas). É o que permite comparar por cosseno.

Um caso junta as **três coisas** de que o sistema precisa: *o que se disse, o que aconteceu e como comparar.*

Como nasce um caso (alinhamento + impacto, sem lookahead)



- Mede-se a partir do **fecho** (não da abertura): não se credita um salto já no preço de abertura.
- Só se olha para a **frente** (anti-lookahead): é *evidência* do que aconteceu, não previsão.
- Sem preços alinhados, ou janela fora dos dados $\Rightarrow$ a notícia é descartada (não inventada).

Já sabes **o problema**, **o sistema** e **os dados**.

A seguir: *o código módulo a módulo* (linha a linha) e o *workflow ponta a ponta* com exemplos reais.

O código num relance

- **Um pacote por componente** (mercado, deteção, correlação, base de conhecimento, explicação, entrega). Cada um faz *uma* coisa.
- **Lógica pura separada de I/O**: os cálculos (z-score, cosseno, impacto) não precisam de rede nem do modelo pesado $\Rightarrow$ testam-se offline e rápido.
- **Programado a interfaces**: o Embedder é abstrato, com 2 implementações trocáveis.

Nos slides seguintes: cada módulo com um *snippet simplificado mas fiel*, linha a linha, e um exemplo concreto de valores.

Detetor de anomalias — o z-score

```
def detect_latest(returns, window=20, threshold=3.0):
    recent = returns[-window-1 : -1]      # 20 dias ANTERIORES (sem hoje)
    mu, sigma = mean(recent), std(recent)
    z = (returns[-1] - mu) / sigma         # quantos desvios-padrao e' hoje
    return AnomalyResult(is_anomaly = abs(z) > threshold,
                        z_score=z, mean=mu, std=sigma, window=window, ...)
```

- **recebe** a série de retornos; **devolve** um AnomalyResult (decisão + todas as quantidades).
- **recent**: só dias *antes* de hoje $\Rightarrow$ **sem lookahead**.
- **Exemplo (TSLA)**: $\mu = -0,92\%$, $\sigma = 2,73\%$, hoje = $+19,82\% \Rightarrow z = +7,61 \Rightarrow$ anomalia.

Similaridade do cosseno + top- k

```
def cosine_similarities(query, matrix):      # query: 1 vetor; matrix: N vetores
    return (matrix @ query) / (row_norms(matrix) * norm(query))
```

```
def top_k_similar(query, matrix, k=5):
    sims = cosine_similarities(query, matrix)
    return indices_of_largest(sims, k)      # os k mais parecidos (+ score)
```

- **recebe** o vetor da notícia nova e a matriz de vetores da KB; **devolve** os índices dos **k mais semelhantes** e o seu score (0 a 1).
- `matrix @ query` é o produto interno; dividir pelas normas dá o **cosseno** (o ângulo).
- **Exemplo:** para a consulta da Nvidia, os 5 maiores scores apontam para notícias de chips de IA.

Event study — o impacto pós-evento

```
def post_event_returns(close, event_idx, horizons=(1,3,5)):
    p0 = close[event_idx]                # fecho do dia do evento
    return { h: close[event_idx + h]/p0 - 1    # retorno acumulado a +h dias
            for h in horizons }
```

- **recebe** a série de fechos e a posição do evento; **devolve** {1:..., 3:..., 5:...}.
- Mede sempre **para a frente**, a partir do **fecho** (p_0). Se faltar dados $\Rightarrow$ NaN (não inventa).
- **Exemplo (AAPL)**: +1d = +0,45%, +3d = +2,50%, +5d = +4,45%.

Embedder — uma interface, duas implementações

```
class Embedder(Protocol):          # "contrato": tem dim e encode()
    dim: int
    def encode(self, texts) -> matriz (n, dim): ...

class HashingEmbedder:             # baseline SEM dependencias (reprodutivel, p/ testes)
class SbertEmbedder:               # SBERT pre-treinado (a abordagem da tese, inferencia)
```

- Ambos cumprem o mesmo contrato $\Rightarrow$ a KB e a recuperação funcionam com qualquer um.
- Por isso podemos **comparar** MiniLM vs MPNet vs lexical de forma **justa** (só troca o embedder).
- O HashingEmbedder permite correr tudo **sem a stack pesada** (torch).

Base de conhecimento — construir e recuperar

```
class HistoricalKB:
    def build(news, prices, embedder):          # 1 "caso" por noticia
        for (date, ticker, headline) in news:
            e = first_trading_day >= date      # alinhar ao evento
            impacts = post_event_returns(prices[ticker], e)
            emb      = embedder.encode([headline])[0]
            add NewsRecord(date, ticker, headline, impacts, emb)

    def find_precedents(query_text, embedder, top_k=5):
        q = embedder.encode([query_text])[0]
        return top_k_similar(q, all_embeddings) # -> [(NewsRecord, score), ...]
```

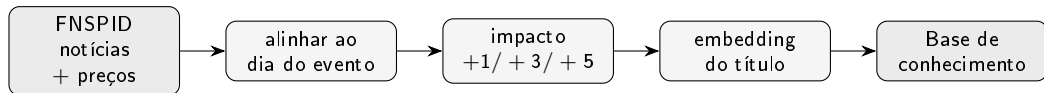
build (offline) cria a memória de casos; **find_precedents** (live) devolve os mais parecidos com a notícia nova. É o **núcleo** do sistema (raciocínio baseado em casos).

Motor de explicação — o alerta *fiel*

```
def explain_news_impact(ticker, headline, precedents, horizon=1):
    avg = mean(impacto[horizon] de cada precedente) # so a media
    linhas = [f"impacto medio {horizon}d: {avg:.2%}"]
    for (rec, score) in precedents: # mostra CADA precedente
        linhas += [f"{rec.date} {rec.ticker} (sim {score:.2f}) {rec.impactos[...]:.2%}"]
    linhas += ["Nota: resultado passado, NAO e' previsao."]
    return "\n".join(linhas)
```

- O texto é montado **a partir dos próprios objetos calculados** $\Rightarrow$ **fiel por construção** (não pode divergir da lógica).
- Mostra **cada** precedente (não só a média) e termina sempre com o **aviso de não-previsão**.

Workflow (1/3): construir a memória, uma vez (offline)



Resultado: uma base de **casos** (*notícia* $\rightarrow$ *impacto* + *vetor*), pronta a ser consultada. Feita **uma vez**; depois é só usar.

Workflow (2/3): gatilho de mercado — TSLA, 24-10-2024 (real)

- 1 Preços live $\rightarrow$ log-return de hoje: $r = +19,82\%$ (movimento de $\approx +22\%$).
- 2 Janela dos 20 dias anteriores: $\mu = -0,92\%$, $\sigma = 2,73\%$.
- 3 $z = (19,82 - (-0,92))/2,73 = +7,61$.
- 4 $|7,61| > 3 \Rightarrow$ **anomalia**. O alerta expõe r, μ, σ , janela e limiar, e traduz: " $\approx 7,6 \times$ a oscilação diária típica desta ação".

Porque é convincente

Reprodutível por `scripts/evaluate_anomaly.py` (janela fixa). O mesmo valor sai sempre.

Workflow (3/3): gatilho de notícia — Nvidia (real)

Consulta: *"Nvidia raises guidance on surging demand for AI data-centre accelerators"* → embedding → cosseno top-5 → alerta:

Alerta NVDA | impacto medio 1 dia: -1,97%

- MSFT (sim 0.68) +1d -3,46% "Qualcomm debuts AI data center chips..."
- NVDA (sim 0.68) +1d -1,64% "Qualcomm debuts AI data center chips..."
- META (sim 0.68) +1d -2,65% "Qualcomm debuts AI data center chips..."
- GOOGL (sim 0.64) +1d -0,46% "...bigger than Nvidia..."
- NVDA (sim 0.58) +1d -1,64% "...NVIDIA..."

Nota: precedentes por semelhança; impacto observado, NAO previsao.

Todos falam de **chips de IA para data-center**, mesmo vindo de empresas diferentes (**cross-ticker**): a recuperação é por **significado**, não pelo nome da empresa.

A nota mais honesta (e a pergunta certa do júri)

Um título *positivo* deu precedentes *negativos* ($-1,97\%$). Engana?

A semelhança capta o **tema** ("chips de IA"), **não a direção** (o sentimento). Por isso um título positivo pode recuperar um *cluster* de ameaça competitiva, com desfechos negativos.

A média é **evidência sobre um tema**, **não uma previsão** para este caso. É por isso que o alerta:

- mostra sempre os precedentes **um a um** (não só a média);
- termina com o **aviso de não-previsão** (evita a *sobre-confiança*).

Melhorias futuras: de-duplicar precedentes e **sinalizar quando o cluster discorda na direção**.

Já viste **o código** e o **workflow** com exemplos reais.

A seguir: *correr a app* tu mesmo (1 comando), depois *a avaliação*, as *decisões* e as *perguntas do júri*.

Deixar de ser "caixa preta": a app são *duas funções*

Não há magia. A app é dois pontos de entrada (funções que já viste):

- **Gatilho de notícia** — corre **offline**, com a base de conhecimento de amostra que está no repositório. **Não precisa de chaves nem de internet**. É o melhor para experimentar.
- **Gatilho de mercado** — busca preços ao vivo (precisa de internet); não envia nada.

O mais importante

Fiz-te um comando único que corre os dois e mostra o resultado, **sem configurar nada**: `python scripts/demo.py`. Vê o próximo slide.

Passo 1: um comando, e vê a app a funcionar

A partir da pasta do projeto:

```
$ python scripts/demo.py
```

```
==== GATILHO DE NOTICIA (offline, base de conhecimento de amostra) ====
```

```
Alerta NVDA | impacto medio 5 dias: +6,46%
```

- 2023-05-25 NVDA (sim 0.60) +5d +3,55% "Nvidia guidance surges..."
- 2023-04-25 MSFT (sim 0.38) +5d +10,89% "Microsoft cloud growth..."
- 2023-06-13 NVDA (sim 0.38) +5d +4,93% "Nvidia unveils new AI..."

```
Nota: impacto observado, NAO e' previsao.
```

```
==== GATILHO DE MERCADO (precos ao vivo; nao envia) ====
```

```
No anomaly for AAPL today (z-score +0.89, within +-3).
```

Nada é enviado. O gatilho de notícia dá *sempre* o mesmo resultado (determinístico).

Passo 1 explicado: o que aconteceu, linha a linha

- 1 A consulta *“Nvidia demand surges on AI chip orders”* virou um **vetor** (embedding).
- 2 Comparou-se por **cosseno** com todos os casos da base → os **3 mais parecidos**.
- 3 Para cada um, leu-se o **impacto +5d** já guardado (evidência do passado).
- 4 Média = +6,46% (o mesmo +6,5% do exemplo do Cap. 3!), com o **aviso de não-previsão**.

É *exatamente* o pipeline dos slides anteriores — agora a correr no teu computador.

Passo 2: mudar um parâmetro e ver mudar (o "e se?")

Mesma consulta, mas `top_k=5`, `horizon=1`:

Alerta NVDA | impacto medio 1 dia: +3,40%

- NVDA (sim 0.60) +1d +2,54% "Nvidia guidance surges..."
- MSFT (sim 0.38) +1d +7,24% "Microsoft cloud growth..."
- NVDA (sim 0.38) +1d +4,81% "Nvidia unveils new AI..."
- JPM (sim 0.31) +1d +1,95% "JPMorgan acquires..." <- menos parecido
- AAPL (sim 0.25) +1d +0,45% "Apple expands services..." <- menos parecido

O que mudou: `top_k=5` $\Rightarrow$ 5 precedentes (os 2 últimos com semelhança *baixa*); `horizon=1` $\Rightarrow$ olha a +1 dia (média +3,40%). Assim respondes a "e se aumentasse o *k*?" com um facto, não um palpite.

Passo 3: ver que tudo funciona (os testes)

```
$ bash scripts/verify.sh
..... [100%] 47 passed
All checks passed! (ruff: estilo OK)
```

- Cada peça (z-score, cosseno, event study, KB, explicação) tem **testes automáticos** que correm **offline**, rápido.
- Se o júri perguntar "como sei que não inventaste?", a resposta é: **os testes** e os **scripts de avaliação** reproduzem os números.

E o Telegram / notícias ao vivo?

- Enviar para o **Telegram** e buscar **notícias ao vivo** precisa de chaves (gratuitas) num ficheiro `.env` (nunca versionado). Passos exatos: `docs/design/how_to_run.md`.
- Mas **não precisas disso para estudar**: a demo acima já prova o caminho todo, offline.
- **Dica (Windows)**: a consola pode rebentar com os emojis do alerta; a demo já força UTF-8 para isso não acontecer.

Resumo de "como corro a app"

```
python scripts/demo.py (ver a app) · bash scripts/verify.sh (ver que funciona) · docs/design/how_to_run.md  
(tudo o resto, incluindo Telegram).
```

Mais um exemplo: quando o baseline *falha* (e porquê)

Consulta de **banca**, com o encoder *baseline* (só palavras) da demo:

```
run_news_trigger('JPM', 'Bank profits jump as interest rates rise', top_k=3)
```

Alerta JPM | impacto medio 5 dias: -2,99%

- AAPL (sim 0.25) +5d +4,45% "Apple expands services..."
- JPM (sim 0.15) +5d +1,44% "JPMorgan acquires failed bank..."
- TSLA (sim 0.14) +5d -14,86% "Tesla margins narrow..."

Os scores são **baixos** e os precedentes **maus**. Porquê? O baseline só vê **sobreposição de palavras**: "Bank profits...interest rates" não partilha palavras com "JPMorgan...banking turmoil". É o **problema de vocabulário** — e é *exatamente* isto que o **SBERT** resolve (capta *significado*, não palavras iguais). Por isso a versão real da tese usa SBERT.

Constrói a tua própria base de conhecimento

Queres os teus exemplos? Constrói uma KB a partir de um CSV `date,ticker,headline`:

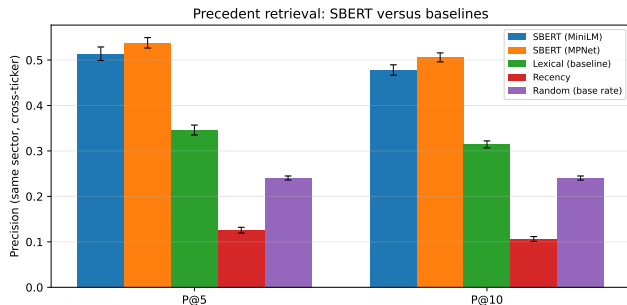
```
# baseline (rapido, offline):  
python scripts/build_kb.py --news data/finnhub_news.csv --out data/kb.jsonl  
# SBERT (significado real; precisa da stack pesada de ML):  
python scripts/build_kb.py --news ... --out data/kb_sbert.jsonl --sbert
```

- Para cada título: alinha ao 1.º dia de negociação, busca preços, mede o impacto +1/ +3/ +5 e gera o embedding ⇒ **um caso por título**.
- Consulta depois com `run_news_trigger(..., kb_path=..., embedder=...)`.
- **Regra:** consulta com o *mesmo* embedder com que construístes (mesma dimensão/modelo).

Como avaliamos — sem nos enganarmos

- O plano foi **fixado antes** de correr qualquer experiência (uma mini "pré-registo"): não se escolhe a métrica depois para favorecer o resultado.
- **Recuperação:** $\text{precision}@k$ (acerto nas k melhores), contra **3 baselines** (aleatório, recência, lexical), média de **5 seeds**.
- **Anomalias:** argumento **sem rótulo** (consistência da taxa de disparo entre ações); o F_1 contra um proxy é só *suporte*.
- **Honestidade:** sem previsão nem trading $\Rightarrow$ métricas de lucro estão *fora de âmbito* por desenho.

Recuperação: SBERT bate todas as baselines

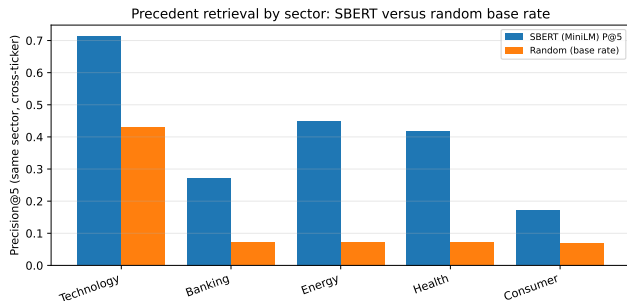


P@5 (cross-ticker):

- SBERT-MiniLM **0,514**
- SBERT-MPNet **0,538**
- lexical 0,346
- aleatório 0,240
- recência 0,126

Leitura: $\approx 2,1 \times$ o acaso e bem acima do lexical
 $\Rightarrow$ o valor vem do *significado*, não de palavras iguais. Desvios pequenos ($\sim 0,01$) $\Rightarrow$ robusto.

Recuperação por setor: onde acrescenta mais

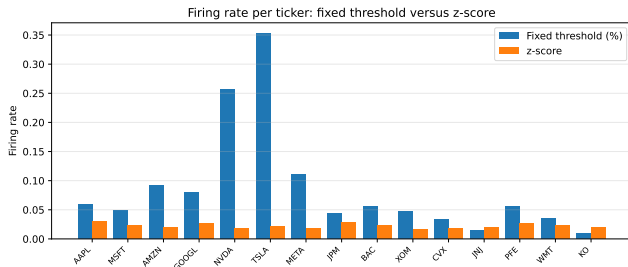


O justo é o **lift** (ganho sobre o acaso), não o valor bruto:

- energia **+0,377**, saúde **+0,348** (vocabulário distintivo)
- consumo **+0,100** (notícias genéricas)

A tecnologia tem P@5 bruta alta (0,712) só porque **domina o corpus** (taxa-base 0,429).

Anomalias: consistente em todo o mercado



O limiar fixo dispara $\sim 1\%$ numa ação calma e $\sim 35\%$ numa volátil; o **z-score** fica $\sim 2\text{--}3\%$ em *todas*.

Amplitude da taxa: **0,015** (z-score) vs **0,344** (fixo) $\Rightarrow > 20\times$ mais consistente. Este é o argumento **sem rótulo**. O F_1 (0,516 vs 0,218) é suporte.

E desafiámos a regra com um detetor **aprendido** (Isolation Forest, causal, mesma informação): perdeu — F_1 **0,271 vs 0,530**. A escolha simples fica validada por *comparação*.

Triagem de materialidade — o modelo que EU treinei (RQ4)

A pergunta que o modelo responde

Chegam dezenas de notícias por dia. **Quais merecem alerta?** O modelo estima $P(\text{segue-se um movimento ANORMAL})$ — *materialidade*, nunca direção nem preço.

- **Rótulo (sem anotar à mão):** material se $|\text{retorno do ticker} - \text{retorno do SPY}| \geq 2\%$ nos 3 dias úteis *depois* do evento — calculado pelo NOSSO event study.
- **Features (só passado):** volatilidade dos 20 dias *antes*, momentum 5d, retorno do próprio dia, comprimento do título, setor, embedding SBERT do título.
- **Anti-batota testado:** um teste unitário *muda os preços do futuro* e verifica que as features NÃO mudam (e o rótulo sim). É a resposta pronta à pergunta nº 1 do júri.

Corpus: FNSPID 2018–2023, **79 753** exemplos, 14/15 tickers (a Meta é "FB" no corpus).

Como se treina sem batota — e como se mede

- **Split temporal** por dias únicos (70/15/15): treino no passado, teste no futuro — nunca ao contrário. **Embargo** de 5 dias entre blocos (nenhuma janela de rótulo atravessa).
- **Calibração (Platt)**: uma sigmóide ajustada SÓ na validação transforma scores em *probabilidades honestas* ("70%" acontece ~70% das vezes).
- **6 famílias**: alertar-sempre (chão) · LR só-volatilidade (a baseline decisiva) · LR contexto/texto/ambos · gradient boosting (teto).

As métricas, em palavras

PR-AUC: qualidade do ranking com classes desequilibradas; o chão é a prevalência.

Precisão@5/dia: das 5 melhores por dia (o que um humano aguenta), quantas foram materiais — mede a *fadiga de alertas*.

Brier: erro quadrático das probabilidades (menor = mais honesto).

Pré-compromisso: se o modelo não bater a baseline de volatilidade, **reporta-se na mesma**.

O resultado honesto — e porque é BOM para a defesa

Modelo	PR-AUC	P@5/dia
Alertar-sempre	0,378	0,163
LR só-volatilidade	0,542	0,632
LR só-contexto	0,538	0,632
LR contexto+texto	0,496	0,585
Gradient boosting	0,469	0,551
LR só-texto	0,439	0,572

- **Como mecanismo, funciona:** dentro de 5 alertas/dia, a precisão quase **quadruplica** (0,632 vs 0,163), com probabilidades calibradas (Brier 0,218 vs 0,622).
- **O veredicto:** nenhum modelo que lê o texto bateu a volatilidade $\Rightarrow$ o sinal está no *contexto de mercado*, não nas palavras do título.
- É a **2.ª** comparação justa "aprendido vs simples" que a escolha transparente vence (a 1.ª foi IF vs z-score) — honestidade que *fortalece* a tese.

E depois do deploy? O loop de pós-validação (a minha ideia de "RL", na forma defensável)

Cada decisão fica registada; dias depois, quando a janela fecha, é rotulada com o que *realmente* aconteceu $\Rightarrow$ precisão/calibração ao vivo + dados novos para retreinar. Não é RL clássico (os alertas não movem o mercado) — é aprendizagem contínua com rótulos atrasados.

Ameaças à validade (ditas antes que perguntem)

- **Construção:** relevância por *setor* é um proxy; o rótulo de anomalia é volatilidade-relativo (por isso a consistência sem-rótulo tem primazia).
- **Interna:** match trivial é removido (cross-ticker); sem lookahead; resta o *confundimento* (mercado global), que as janelas curtas limitam.
- **Externa:** 15 ações grandes, US, janela recente $\Rightarrow$ não generaliza a small-caps/outras mercados sem mais dados.
- **Estatística:** 5 seeds, desvios reportados; gap grande face ao desvio, mas é amostra modesta.

Decisões de desenho: porquê isto e não outra coisa

Decisão	Escolhido	Alternativa	Porquê (rejeição)
Deteção	z-score (rolling)	Isolation Forest, deep	transparência; sinal 1-D não justifica opacidade
Recuperação	SBERT + cosseno	léxico, word2vec, LLM	significado, barato, reproduzível
Impacto	event study (retorno <i>raw</i>)	CAR (ajustado)	observável; sem modelo opaco a explicar
Explicação	transparente + precedentes	só LIME/SHAP	fiel por construção, não aproximação
Avaliação	precision@k <i>cross-ticker</i>	—	padrão; evita match trivial pelo nome

Princípio único: **simplicidade defensável**. Onde duas opções empatam, escolhe-se a mais *transparente*, porque o sistema é para um não-especialista.

"E se mudássemos...?" (sensibilidade dos parâmetros)

- **Janela do z-score** (10/20/60 dias): F_1 vs proxy = 0,385/0,516/0,678. Janela maior $\Rightarrow$ baseline de volatilidade mais estável (mas acaba por saturar).
- **Limiar k** (=3): maior $\Rightarrow$ menos alertas (mais seletivo); menor $\Rightarrow$ mais alertas (mais ruído).
- **top- k** (=5): quantos precedentes mostrar; um alerta só comporta uns poucos.
- **Horizonte** (+1/ +3/ +5): janelas curtas limitam o confundimento de mercado; mais longo $\Rightarrow$ mais ruído alheio à notícia.

Saber estes *trade-offs* é a melhor resposta a "e se aumentasse este valor?".

O produto, HOJE — o que está ao vivo (e como mostrar)

Não é só uma tese em papel: o sistema **corre** em produção, de graça, sem servidor próprio.

- **Dashboard público** — `investigator.streamlit.app`: os dois gatilhos e a avaliação, no browser. (Usa o embedder *baseline* sobre uma fatia curada de 2.016 manchetes FNSPID 2018–2023; o SBERT e o seu ganho medido estão na página Evaluation.)
- **Canal Telegram + timer** — o GitHub Actions varre a watchlist seg–sex após o fecho US e publica alertas explicados no canal. Anti-repetição de notícias (≤ 2 dias) e anti-duplicado em feriados (só avalia com sessão nova).
- **Bot interativo** — `/watch` TSLA, `/list`, `/stop`: cada utilizador gere a sua watchlist (SQLite local; *long-polling*, sem servidor). Off por defeito no runner; fail-open (nunca parte a produção).
- **À prova de wifi** — se a demo ao vivo falhar: `python scripts/demo.py` é offline e determinística (reproduz o +6,46% do Cap. 3).

Uma frase para o júri: "o desenho de produção é conservador: tudo o que é opcional está desligado por defeito e falha aberto — a produção nunca depende de um componente opcional estar vivo."

Perguntas difíceis do júri (1/2)

Isto não é só usar bibliotecas?

Sim, e numa tese de *Engenharia* de IA é isso que se pede: a contribuição é a integração, a metodologia notícia→impacto e a avaliação honesta. O núcleo é **CBR** (raciocínio baseado em casos), não ad hoc.

Porquê não um LLM / deep learning?

Defensibilidade: para uma série de retornos e títulos curtos, o ganho não compensa a perda de transparência e o custo. LLMs ficam como trabalho futuro (infraestrutura pronta: FAISS para escala).

A precision@5 de ~0,51 é boa?

É $\sim 2,1\times$ o acaso e acima do lexical, com desvio pequeno (5 seeds). É honesta e **preliminar** (corpus recente; FNSPID multi-ano é o passo seguinte).

O proxy de setor não é fraco?

É automático, sim — por isso é limitação explícita e por isso dou primazia ao argumento **sem rótulo**.

Perguntas difíceis do júri (2/2)

Como garantem que não há lookahead?

O z-score usa só dias anteriores; o impacto mede-se só *depois* do evento, do fecho do 1.º dia de negociação $\geq$ data. Está em código e testado.

Como sei que a explicação é verdadeira?

É renderizada dos mesmos objetos calculados $\Rightarrow$ **fiel por construção**; um teste automático confirma que reproduz exatamente os precedentes, sem inventar.

Um título positivo deu precedentes negativos. Engana?

A semelhança capta **tema, não direção**; a média é evidência sobre um tema, não previsão. Mostro os precedentes um a um e aviso que não é previsão (evita sobre-confiança).

Usaram IA para escrever a tese?

Sim, declarado honestamente; dirigi o trabalho, revi e validei tudo; as decisões e o conteúdo são da minha responsabilidade.

Checklist final: defender com calma

Consigo responder a tudo isto?

- O que é a tese? (3 frases)
- Porque importa o problema?
- Porquê esta metodologia?
- O que acontece em cada fase?
- Porque o código faz isto?
- E se mudasse este parâmetro?
- Porque bate as baselines?
- Limitações e trabalho futuro?

Defesa em 3 frases (decorar)

Construí um sistema que avisa o investidor de retalho e **explica** cada alerta com precedentes reais. A contribuição é de **engenharia de IA**: integrar, aplicar e avaliar componentes num todo explicável e reproduzível. **Não prevejo**; mostro evidência do passado, com honestidade sobre os limites.

Onde continuar a estudar (mapa dos materiais)

Este guia não vive sozinho. Cada peça abre uma porta diferente:

- **Ver a app:** `python scripts/demo.py` — os dois gatilhos, offline, num comando.
- **Correr tudo o resto:** `docs/design/how_to_run.md` (Telegram, notícias ao vivo, construir a KB) — começa no §0.0.
- **A tese:** `thesis/main.pdf` — o documento que vais defender (os números vêm daqui).
- **Slides da defesa (EN):** `slides/main.pdf` — a versão curta para o dia.
- **Caderno de defesa (PT):** `docs/defence/caderno_de_defesa.md` — respostas em prosa, número → script → tese.

Sugestão de estudo

Lê este guia → corre `demo.py` → relê o capítulo da tese sobre a parte que correste → pratica as **perguntas do júri**.
Repete nas partes que ainda parecem "caixa preta".

Agora conheces a tese **de fio a pavio**.

Problema · bases de IA · dados · código · workflow · avaliação · decisões · defesa.

Estuda a par da tese e dos slides; volta às partes que precisares. Boa defesa.